learned about in chapter 8). As usual, you can customize it via configuration properties.

Spring Cloud Stream defines several defaults to make the interaction more resilient, including error channels and graceful shutdown. You can configure different aspects of message processing, including dead-letter queues, acknowledgment flows, and republishing messages on error.

RabbitMQ itself has several features in place to improve reliability and resilience. Among other things, it guarantees that each message is delivered at least once. Be aware that consumers in your applications might receive the same message twice, so your business logic should know how to identify and handle duplicates.

I won't go further into the details, since this is an extensive subject that would require several dedicated chapters to cover it adequately. Instead, I encourage you to read the documentation for the different projects involved in your event-driven architecture: RabbitMQ (https://rabbitmq.com), Spring AMQP (https://spring.io/projects/spring-amqp), and Spring Cloud Stream (https://spring.io/projects/spring-cloud-stream). You can also check out the event-driven patterns described in Sam Newman's *Building Microservices* (O'Reilly, 2021) and Chris Richardson's *Microservices Patterns* (Manning, 2018).

In the last part of the chapter, you'll work with suppliers and consumers and complete the order flow for the Polar Bookshop system.

10.5 Producing and consuming messages with Spring Cloud Stream